Java Array Techniques

Ten problems for recognising the technique behind an array problem

Each problem is written so you meet a technique for the first time and learn to spot its pattern. Read the story, decide which idea makes it fast enough, then solve it. Every problem is on the judge with the same samples and limits; the large hidden tests reject the slow, obvious approach.

The **answer key** starts after the last problem, on separate pages. Try each problem before turning to it.

#	Problem	Difficulty	Time limit
1	The Library Ledger	Easy	3 s
2	The Coffee Machine	Easy	2 s
3	The Balanced Pair	Medium	2 s
4	The Hottest Week	Easy	2 s
5	The Budget Streak	Medium	2 s
6	The Slow Printer	Medium	2 s
7	The Lucky Stretch	Easy	2 s
8	The Letter Detective	Easy	2 s
9	The Ranking Game	Medium	2 s
10	The Coldest Window	Hard	3 s

Submissions are in Java 17 with a `public class Solution`. Input arrives on standard input, and output is compared line by line (trailing spaces are ignored).

PROBLEM 1

The Library Ledger

Time limit 3 s · Memory 256 MB · Easy

A librarian records how many books are borrowed each day for `n` days. Visitors keep asking: *"How many books were borrowed from day L to day R?"* The librarian is tired of recounting every time.

Input

- Line 1: `n`, the number of days
- Line 2: `n` integers `a[0..n-1]`, books borrowed each day
- Line 3: `q`, the number of queries
- Next `q` lines: `L R` (0-indexed, inclusive)

Output

For each query, print the total number of books borrowed from day `L` to day `R`, one per line.

Constraints

- $1 \le n, q \le 10^5$
- $0 \le a[i] \le 10^4$
- $0 \le L \le R < n$

Sample

SAMPLE INPUT

```
5
3 1 4 1 5
3
0 2
1 3
0 4
```

SAMPLE OUTPUT

```
8
6
14
```

Hint. Answering every query with a loop is O(n·q), which is too slow. Precompute cumulative sums once.

Fast I/O. The input can hold hundreds of thousands of numbers. `Scanner` and one `println` per answer are too slow even with the right technique: read with `BufferedReader` + `StringTokenizer` and print once from a `StringBuilder`.

PROBLEM 2

The Coffee Machine

Time limit 2 s · Memory 256 MB · Easy

A coffee machine gives free refills. The manager runs several promotions: *"From day L to day R, add V extra cups per day."* After all promotions, the manager wants the total number of extra cups for each day. Every day starts at 0.

Input

- Line 1: `n u`, the number of days and the number of promotions
- Next `u` lines: `L R V` (0-indexed, inclusive)

Output

One line with `n` integers: the total for each day, separated by spaces.

Constraints

- $1 \le n, u \le 10^5$
- $0 \le L \le R < n$
- $0 \le V \le 10^4$

Sample

SAMPLE INPUT

```
5 3
0 2 3
1 4 2
2 2 5
```

SAMPLE OUTPUT

```
3 5 10 2 2
```

Hint. Applying each promotion day by day is O(n·u). Record only where each promotion *starts* and where it *stops*.

Fast I/O. The input can hold hundreds of thousands of numbers. `Scanner` and one `println` per answer are too slow even with the right technique: read with `BufferedReader` + `StringTokenizer` and print once from a `StringBuilder`.

PROBLEM 3

The Balanced Pair

Time limit 2 s · Memory 256 MB · Medium

A teacher has her students' exam scores, sorted from lowest to highest. She wants to find **two students whose scores add up exactly to a target**.

Input

- Line 1: `n`, the number of students
- Line 2: `n` integers `a[0..n-1]`, sorted in ascending order
- Line 3: `target`, the desired sum

Output

Two indices `i j` with `i < j` and `a[i] + a[j] = target`, or `-1 -1` if no such pair exists. The tests guarantee there is **at most one** such pair.

Constraints

- $2 \le n \le 10^5$
- $0 \le a[i] \le 10^9$, sorted ascending
- $0 \le target \le 2 \cdot 10^9$

Sample

SAMPLE INPUT

```
5
10 25 40 55 90
100
```

SAMPLE OUTPUT

```
0 4
```

Hint. The array is sorted, so don't try every pair. Start with one pointer at each end and move them toward each other.

PROBLEM 4

The Hottest Week

Time limit 2 s · Memory 256 MB · Easy

A weather station records the temperature for `n` days. Find the **maximum total temperature** over any `k` consecutive days.

Input

- Line 1: `n k`
- Line 2: `n` integers `a[0..n-1]`

Output

The maximum sum of any `k` consecutive values.

Constraints

- $1 \le k \le n \le 10^5$
- $-10^4 \le a[i] \le 10^4$

Sample

SAMPLE INPUT

```
7 3
2 -1 5 4 -3 6 1
```

SAMPLE OUTPUT

```
8
```

Hint. Don't add up each window from scratch. Slide the window: add the new day and subtract the one that drops out.

PROBLEM 5

The Budget Streak

Time limit 2 s · Memory 256 MB · Medium

A student tracks his daily expenses. He wants the **longest streak of consecutive days** whose total expense does not exceed his budget `S`.

Input

- Line 1: `n S`
- Line 2: `n` integers `a[0..n-1]`, all non-negative

Output

The length of the longest such streak (0 if no single day fits the budget).

Constraints

- $1 \le n \le 10^5$
- $0 \le a[i] \le 10^4$
- $0 \le S \le 10^9$

Sample

SAMPLE INPUT

```
6 10
4 2 3 7 1 1
```

SAMPLE OUTPUT

```
3
```

Hint. Grow the window with `r`, and shrink it from `l` while its sum is above `S`.

PROBLEM 6

The Slow Printer

Time limit 2 s · Memory 256 MB · Medium

A printer prints x pages per hour. It must finish n print jobs within h hours. It works on one job at a time, and if a job finishes partway through an hour, the rest of that hour is wasted. Find the **minimum speed** x that finishes every job in time.

Input

- Line 1: `n h`
- Line 2: n integers `pages[0..n-1]`

Output

The minimum feasible speed x.

Constraints

- $1 \le n \le 10^5$
- $n \le h \le 10^9$
- $1 \le pages[i] \le 10^9$

Sample

SAMPLE INPUT

```
4 8
3 6 7 11
```

SAMPLE OUTPUT

```
4
```

Hint. "Can every job finish in h hours at speed x?" is monotonic in x: once it is true, it stays true for faster speeds. Binary search on x.

PROBLEM 7

The Lucky Stretch

Time limit 2 s · Memory 256 MB · Easy

A stock trader records each day's profit or loss. Find the **stretch of consecutive days** with the **largest total profit**. The stretch must contain at least one day.

Input

- Line 1: n
- Line 2: n integers `a[0..n-1]` (may be negative)

Output

The maximum sum of a non-empty contiguous subarray.

Constraints

- $1 \le n \le 10^5$
- $-10^4 \le a[i] \le 10^4$

Sample

SAMPLE INPUT

```
9
-2 1 -3 4 -1 2 1 -5 4
```

SAMPLE OUTPUT

```
6
```

Hint. At each day, decide: extend the stretch that ended yesterday, or start fresh today.

PROBLEM 8

The Letter Detective

Time limit 2 s · Memory 256 MB · Easy

Two students want to know whether two words are **anagrams**: whether the letters of one can be rearranged to spell the other.

Input

- Line 1: string `s`
- Line 2: string `t`

Both contain only lowercase letters `a`–`z`.

Output

`YES` if `s` and `t` are anagrams, otherwise `NO`.

Constraints

- $1 \le |s|, |t| \le 10^6$

Sample

SAMPLE INPUT

```
listen
silent
```

SAMPLE OUTPUT

```
YES
```

Hint. Count how many times each letter appears in each string. The counts must match exactly.

PROBLEM 9

The Ranking Game

Time limit 2 s · Memory 256 MB · Medium

A contest gives scores that can be **very large** (up to 10^9 in size). Give each contestant a **rank**: 0 for the smallest score, 1 for the next distinct score, and so on. Contestants with the same score get the same rank.

Input

- Line 1: `n`
- Line 2: `n` integers `a[0..n-1]`

Output

One line with each contestant's rank, in input order, separated by spaces.

Constraints

- $1 \le n \le 10^5$
- $-10^9 \le a[i] \le 10^9$

Sample

SAMPLE INPUT

```
5
100 -5 100 1000000000 7
```

SAMPLE OUTPUT

```
2 0 2 3 1
```

Hint. You can't make an array with 10^9 slots, but you can **map the large values to small ranks** while keeping their order: sort, remove duplicates, then look each value up.

Fast I/O. The input can hold hundreds of thousands of numbers. `Scanner` and one `println` per answer are too slow even with the right technique: read with `BufferedReader` + `StringTokenizer` and print once from a `StringBuilder`.

PROBLEM 10

The Coldest Window

Time limit 3 s · Memory 256 MB · Hard

A weather scientist records each day's minimum temperature. She asks many questions: *"What was the lowest temperature between day L and day R?"* The recorded data never changes.

Input

- Line 1: `n`
- Line 2: `n` integers `a[0..n-1]`
- Line 3: `q`
- Next `q` lines: `L R` (0-indexed, inclusive)

Output

For each query, print the minimum of `a[L..R]`, one per line.

Constraints

- $1 \le n, q \le 10^5$
- $-10^9 \le a[i] \le 10^9$
- $0 \le L \le R < n$

Sample

SAMPLE INPUT

```
6
5 2 8 -1 3 7
4
0 2
2 5
4 4
0 5
```

SAMPLE OUTPUT

```
2
-1
3
-1
```

Hint. The array never changes and there are many range-minimum queries. Precompute a **sparse table** (the minimum of every block whose length is a power of two) so each query takes O(1).

Fast I/O. The input can hold hundreds of thousands of numbers. `Scanner` and one `println` per answer are too slow even with the right technique: read with `BufferedReader` + `StringTokenizer` and print once from a `StringBuilder`.

Answer Key

Model solutions, one per page. Each one passes every test on the judge.

ANSWER 1

The Library Ledger

Technique: **Prefix Sum** · Time limit 3 s · Memory 256 MB · Easy

Why this technique?

You need **many range-sum queries on a static array**. That is the exact fingerprint of prefix sum: build `pref[i+1] = pref[i] + a[i]` once, then every query is `pref[R+1] - pref[L]`.

Complexity

O(n + q) time, O(n) extra space.

Solution (Java 17)

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int n = Integer.parseInt(st.nextToken());
        int[] a = new int[n];
        st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) a[i] = Integer.parseInt(st.nextToken());

        long[] pref = new long[n + 1];
        for (int i = 0; i < n; i++) pref[i + 1] = pref[i] + a[i];

        int q = Integer.parseInt(br.readLine().trim());
        StringBuilder sb = new StringBuilder();
        while (q-- > 0) {
            st = new StringTokenizer(br.readLine());
            int l = Integer.parseInt(st.nextToken());
            int r = Integer.parseInt(st.nextToken());
            sb.append(pref[r + 1] - pref[l]).append('\n');
        }
        System.out.print(sb);
    }
}
```

ANSWER 2

The Coffee Machine

Technique: **Difference Array** · Time limit 2 s · Memory 256 MB · Easy

Why this technique?

Many **range-add updates**, and the result is read **once at the end**. That is the difference-array signature: mark +V where an update starts and -V just after it ends, then one running sum rebuilds every day.

Complexity

O(n + u) time, O(n) extra space.

Solution (Java 17)

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int n = Integer.parseInt(st.nextToken());
        int u = Integer.parseInt(st.nextToken());

        long[] diff = new long[n + 2];
        while (u-- > 0) {
            st = new StringTokenizer(br.readLine());
            int l = Integer.parseInt(st.nextToken());
            int r = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            diff[l] += v;
            diff[r + 1] -= v;
        }

        long running = 0;
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            running += diff[i];
            sb.append(running).append(' ');
        }
        System.out.println(sb);
    }
}
```

ANSWER 3

The Balanced Pair

Technique: **Two Pointers** · Time limit 2 s · Memory 256 MB · Medium

Why this technique?

A **sorted array** plus a **pair condition**, and you need O(n): classic two pointers. If the sum is too small, only moving the left pointer right can increase it; if too large, only moving the right pointer left can decrease it.

Complexity

O(n) time, O(1) extra space.

Solution (Java 17)

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        int n = Integer.parseInt(st.nextToken());

        int[] a = new int[n];
        st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) a[i] = Integer.parseInt(st.nextToken());

        int target = Integer.parseInt(br.readLine().trim());

        int l = 0, r = n - 1;
        int ansL = -1, ansR = -1;
        while (l < r) {
            int sum = a[l] + a[r];
            if (sum == target) { ansL = l; ansR = r; break; }
            else if (sum < target) l++;
            else r--;
        }
        System.out.println(ansL + " " + ansR);
    }
}
```

ANSWER 4

The Hottest Week

Technique: **Sliding Window (fixed size)** · Time limit 2 s · Memory 256 MB · Easy

Why this technique?

A **fixed-size contiguous** block and a **maximum sum**: fixed sliding window. Moving the window one day adds the new day and subtracts the day that drops out, so each window costs O(1).

Complexity

O(n) time, O(1) extra space.

Solution (Java 17)

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        int n = Integer.parseInt(st.nextToken());
        int k = Integer.parseInt(st.nextToken());

        int[] a = new int[n];
        st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) a[i] = Integer.parseInt(st.nextToken());

        long sum = 0;
        for (int i = 0; i < k; i++) sum += a[i];
        long best = sum;

        for (int i = k; i < n; i++) {
            sum += a[i] - a[i - k];
            best = Math.max(best, sum);
        }
        System.out.println(best);
    }
}
```

ANSWER 5

The Budget Streak

Technique: **Sliding Window (variable size)** · Time limit 2 s · Memory 256 MB · Medium

Why this technique?

The **longest contiguous** stretch with a **sum constraint**, and every value is non-negative: variable sliding window. Growing the window can only raise the sum and shrinking can only lower it, so each index enters and leaves the window at most once.

Complexity

O(n) time, O(1) extra space.

Solution (Java 17)

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        int n = Integer.parseInt(st.nextToken());
        long S = Long.parseLong(st.nextToken());

        int[] a = new int[n];
        st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) a[i] = Integer.parseInt(st.nextToken());

        int l = 0, best = 0;
        long sum = 0;
        for (int r = 0; r < n; r++) {
            sum += a[r];
            while (sum > S) {
                sum -= a[l++];
            }
            best = Math.max(best, r - l + 1);
        }
        System.out.println(best);
    }
}
```

ANSWER 6

The Slow Printer

Technique: **Binary Search on the Answer** · Time limit 2 s · Memory 256 MB · Medium

Why this technique?

“Find the **minimum value** such that a **monotonic condition** holds”: binary search on the answer. If speed x finishes in time, every faster speed does too, so the feasible speeds form one range and its smallest value can be found in about 30 checks.

Complexity

O(n log M) time, where M is the largest job; O(1) extra space.

Solution (Java 17)

```
import java.io.*;
import java.util.*;

public class Solution {
    static int[] pages;
    static int h;

    static boolean canFinish(long speed) {
        long hours = 0;
        for (int p : pages) {
            hours += (p + speed - 1) / speed;
            if (hours > h) return false;
        }
        return true;
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        int n = Integer.parseInt(st.nextToken());
        h = Integer.parseInt(st.nextToken());

        pages = new int[n];
        st = new StringTokenizer(br.readLine());
        int max = 0;
        for (int i = 0; i < n; i++) {
            pages[i] = Integer.parseInt(st.nextToken());
            max = Math.max(max, pages[i]);
        }

        long lo = 1, hi = max;
        while (lo < hi) {
            long mid = lo + (hi - lo) / 2;
            if (canFinish(mid)) hi = mid;
            else lo = mid + 1;
        }
        System.out.println(lo);
    }
}
```

ANSWER 7

The Lucky Stretch

Technique: **Kadane's Algorithm** · Time limit 2 s · Memory 256 MB · Easy

Why this technique?

The **maximum sum of a contiguous** subarray, where values may be negative: Kadane. The best stretch ending today either extends yesterday's best stretch or starts fresh today, whichever is larger.

Complexity

O(n) time, O(1) extra space beyond the input.

Solution (Java 17)

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());

        StringTokenizer st = new StringTokenizer(br.readLine());
        long[] a = new long[n];
        for (int i = 0; i < n; i++) a[i] = Long.parseLong(st.nextToken());

        long cur = a[0], best = a[0];
        for (int i = 1; i < n; i++) {
            cur = Math.max(a[i], cur + a[i]);
            best = Math.max(best, cur);
        }
        System.out.println(best);
    }
}
```

ANSWER 8

The Letter Detective

Technique: **Frequency Array** · Time limit 2 s · Memory 256 MB · Easy

Why this technique?

A **small, bounded value range** (26 letters) and the task is **counting occurrences**: a frequency array beats a HashMap. Count up for one string and down for the other; anagrams leave every count at 0.

Complexity

O(|s| + |t|) time, O(26) extra space.

Solution (Java 17)

```
import java.io.*;

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        String s = br.readLine().trim();
        String t = br.readLine().trim();

        if (s.length() != t.length()) {
            System.out.println("NO");
            return;
        }

        int[] freq = new int[26];
        for (int i = 0; i < s.length(); i++) {
            freq[s.charAt(i) - 'a']++;
            freq[t.charAt(i) - 'a']--;
        }

        for (int f : freq) {
            if (f != 0) {
                System.out.println("NO");
                return;
            }
        }
        System.out.println("YES");
    }
}
```

ANSWER 9

The Ranking Game

Technique: **Coordinate Compression** · Time limit 2 s · Memory 256 MB · Medium

Why this technique?

A **huge value range**, a **small n**, and only the **order** matters: compress values to ranks. Sort a copy, remove duplicates, and each value's rank is its position in that list, found by binary search.

Complexity

O(n log n) time, O(n) extra space.

Solution (Java 17)

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());

        int[] a = new int[n];
        StringTokenizer st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) a[i] = Integer.parseInt(st.nextToken());

        int[] sorted = a.clone();
        Arrays.sort(sorted);

        int m = 0;
        for (int i = 0; i < n; i++) {
            if (i == 0 || sorted[i] != sorted[i - 1]) {
                sorted[m++] = sorted[i];
            }
        }

        StringBuilder sb = new StringBuilder();
        for (int x : a) {
            int rank = Arrays.binarySearch(sorted, 0, m, x);
            sb.append(rank).append(' ');
        }
        System.out.println(sb);
    }
}
```

ANSWER 10

The Coldest Window

Technique: **Sparse Table** · Time limit 3 s · Memory 256 MB · Hard

Why this technique?

A **static array**, **many range-minimum queries**, and each must be **O(1)**: sparse table. `table[k][i]` holds the minimum of the 2^k values starting at i. Any range is covered by two overlapping power-of-two blocks, and overlap is harmless for minimum.

Complexity

O(n log n) to build, O(1) per query, O(n log n) space.

Solution (Java 17)

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());

        long[] a = new long[n];
        StringTokenizer tok = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) a[i] = Long.parseLong(tok.nextToken());

        int LOG = 1;
        while ((1 << LOG) <= n) LOG++;

        long[][] table = new long[LOG][n];
        for (int i = 0; i < n; i++) table[0][i] = a[i];
        for (int k = 1; k < LOG; k++)
            for (int i = 0; i + (1 << k) <= n; i++)
                table[k][i] = Math.min(table[k - 1][i], table[k - 1][i + (1 << (k - 1))]);

        int q = Integer.parseInt(br.readLine().trim());
        StringBuilder sb = new StringBuilder();
        while (q-- > 0) {
            StringTokenizer st = new StringTokenizer(br.readLine());
            int l = Integer.parseInt(st.nextToken());
            int r = Integer.parseInt(st.nextToken());
            int k = 31 - Integer.numberOfLeadingZeros(r - l + 1);
            sb.append(Math.min(table[k][l], table[k][r - (1 << k) + 1])).append('\n');
        }
        System.out.print(sb);
    }
}
```